МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«МИРЭА — Российский технологический университет»

РТУ МИРЭА

Институт перспективных технологий и индустриального программирования (ИПТИП)

Кафедра индустриального программирования (ИП)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Программирование корпоративных индустриальных систем»

Практическое задание № 4

Студент группы ЭФМО-02-25 Бондарь А.Р. _____
(подпись)

Доцент Макиевский С.Е. _____
(подпись)

Отчет предоставлен «____»_____ 2026г

Москва 2026

СОДЕРЖАНИЕ

Введение 3
1 Структура библиотек классов 4
1.1 Перечень библиотек и их назначение 4
1.2 Правила зависимостей между библиотеками 5
1.3 UML диаграмма пакетов структуры библиотек 6
1.4 UML диаграммы классов для библиотеки Domain 6
1.5 UML диаграммы классов для библиотеки Application 7
1.6 Описание публичных контрактов библиотек 7
1.7 Тестируемость и изоляция зависимостей 8
1.8 Шаблон структуры репозитория и правила именования 8
1.9 План сопровождения структуры библиотек 9
2 Реализация фрагмента модели в коде 10
2.1 Выбор фрагмента UML модели 10
2.2 Правила сопоставления UML конструкций к C# 10
2.3 Создание структуры проекта 10
2.4 Реализация каркаса классов 11
2.5 Реализация инвариантов и бизнес ограничений 16
2.6 Реализация отношений между объектами 16
2.7 Подготовка модульных тестов 16
2.8 Таблица соответствия UML и кода 18
Заключение 19

ВВЕДЕНИЕ

В данной части проекта на основе ранее разработанных моделей требований, поведения и компонентов (пр.5–10) выполняется проектирование структуры библиотек классов (модулей) и реализация ключевого фрагмента доменной модели на языке C#.

Результат включает:

- UML диаграммы пакетов и классов;
- правила зависимостей и контракты;
- код классов с инвариантами;
- модульные тесты;
- таблицу соответствия UML и кода.

1 СТРУКТУРА БИБЛИОТЕК КЛАССОВ

1.1 Перечень библиотек и их назначение

Таблица 1 – Перечень библиотек и их назначение

Библиотека	**Ответственность**	**Запрещено размещать**
Domain	Сущности, value objects, агрегаты, доменные события, интерфейсы репозиториев (без реализаций), доменные сервисы. Содержит бизнес-логику и инварианты.	Инфраструктурные детали (БД, сеть, файловая система), конкретные реализации репозиториев, зависимости от внешних фреймворков.
Application	Сервисы прикладного уровня (оркестрация сценариев), команды, запросы, DTO, мапперы, интерфейсы для транзакций (Unit of Work), обработчики событий.	Доменная логика, инфраструктура, конкретные реализации репозиториев. Зависит только от Domain и абстракций инфраструктуры.
Contracts	Публичные контракты для внешних интеграций (API DTO, интерфейсы для отправки сообщений). Используется для обмена данными между модулями и внешними системами.	Доменная логика, реализации.
Infrastructure	Реализации репозиториев (Entity Framework, Dapper), Unit of Work, конкретные провайдеры (EmailSender, PaymentGatewayClient), логирование, доступ к файлам.	Доменная логика, прикладные сервисы (кроме вызовов через интерфейсы).
Integration	Адаптеры для внешних систем (платёжный шлюз, SMTP), обработчики интеграционных событий, клиенты REST API.	Бизнес-логика домена.
Presentation	Контроллеры (REST API), фильтры, модели представления (если есть UI), настройка маршрутизации.	Доменная логика, прямые обращения к БД.
SharedKernel	Общие типы, перечисления, базовые классы, исключения, утилиты, используемые несколькими модулями.	Специфичная бизнес-логика.

Все библиотеки, кроме Infrastructure и Integration, должны быть независимы от конкретных СУБД и сетевых протоколов.

1.2 Правила зависимостей между библиотеками

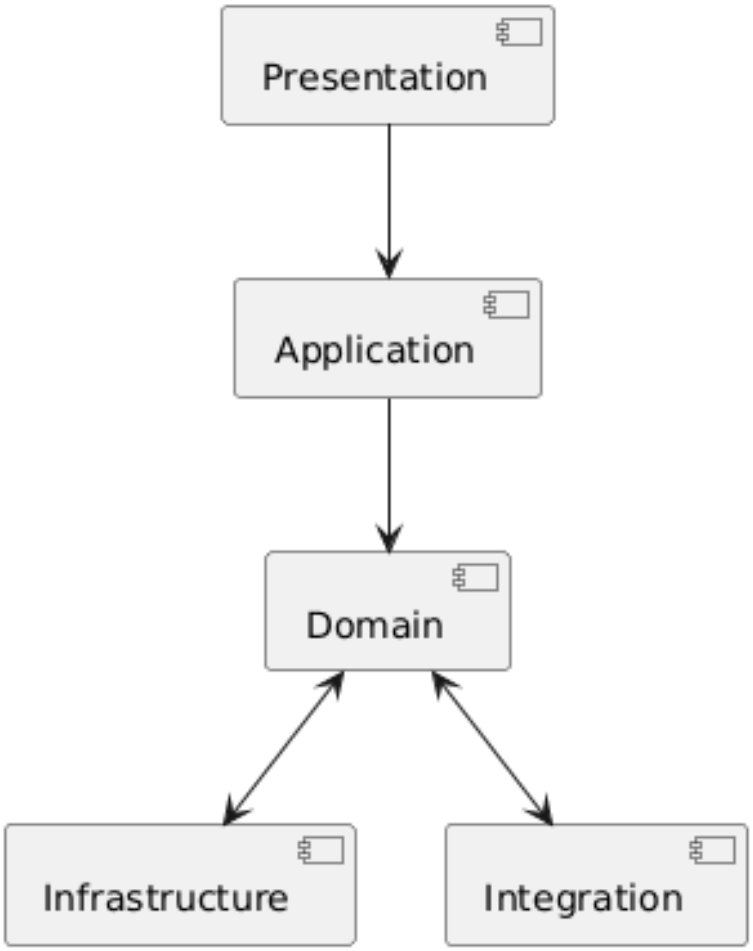

Рисунок 1 – Зависимости между библиотеками

– **Domain** не имеет зависимостей (может зависеть только от SharedKernel, если тот не содержит инфраструктуры);

– **Application** зависит от Domain и Contracts (через интерфейсы), но не от Infrastructure;

– **Infrastructure** и **Integration** зависят от Domain (реализуют интерфейсы репозиториев, сервисов) и могут зависеть от Application только через интерфейсы, но не напрямую;

– **Presentation** зависит от Application (вызывает его сервисы) и Contracts (для DTO);

– Зависимости направлены внутрь домена, циклы запрещены;

– Интерфейсы для репозиториев и инфраструктурных сервисов находятся в Domain (или в отдельном проекте Abstractions, но мы разместим их в Domain для простоты).

1.3 UML диаграмма пакетов структуры библиотек

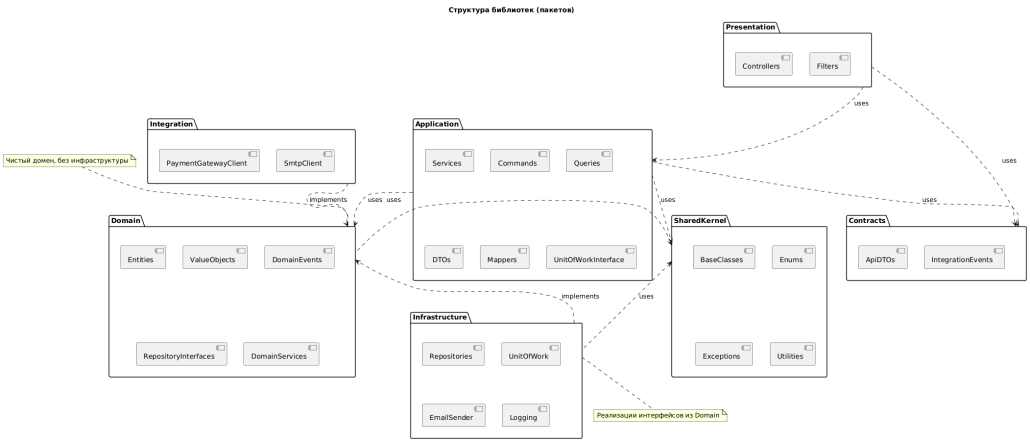

Рисунок 2 – UML диаграмма пакетов

1.4 UML диаграммы классов для библиотеки Domain

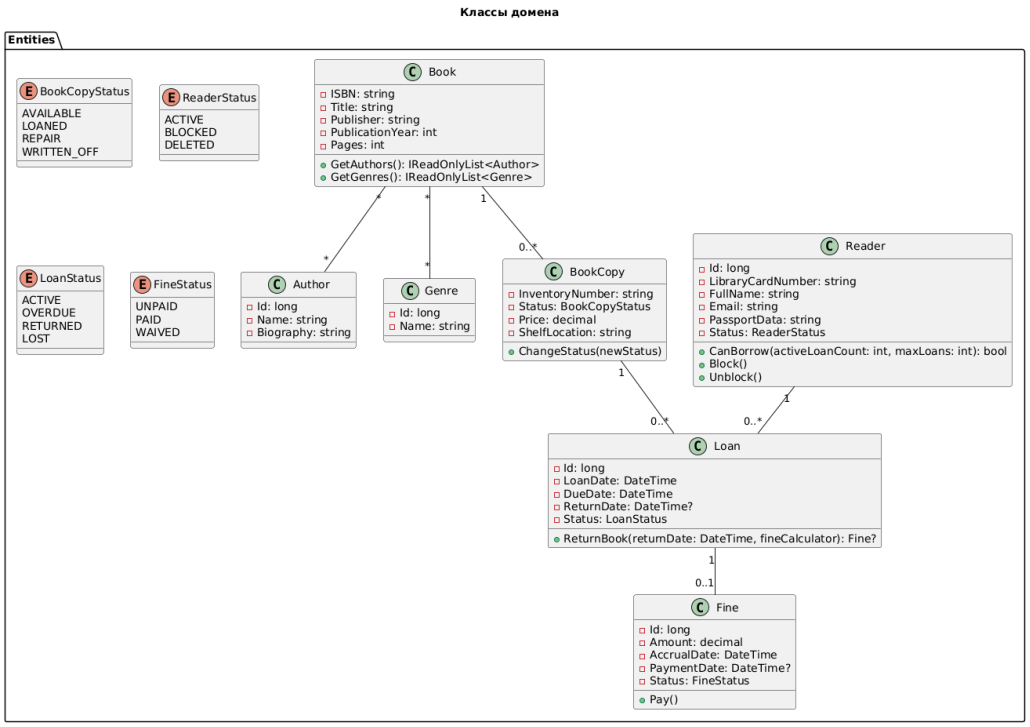

Рисунок 3 – Классы домена

1.5 UML диаграммы классов для библиотеки Application

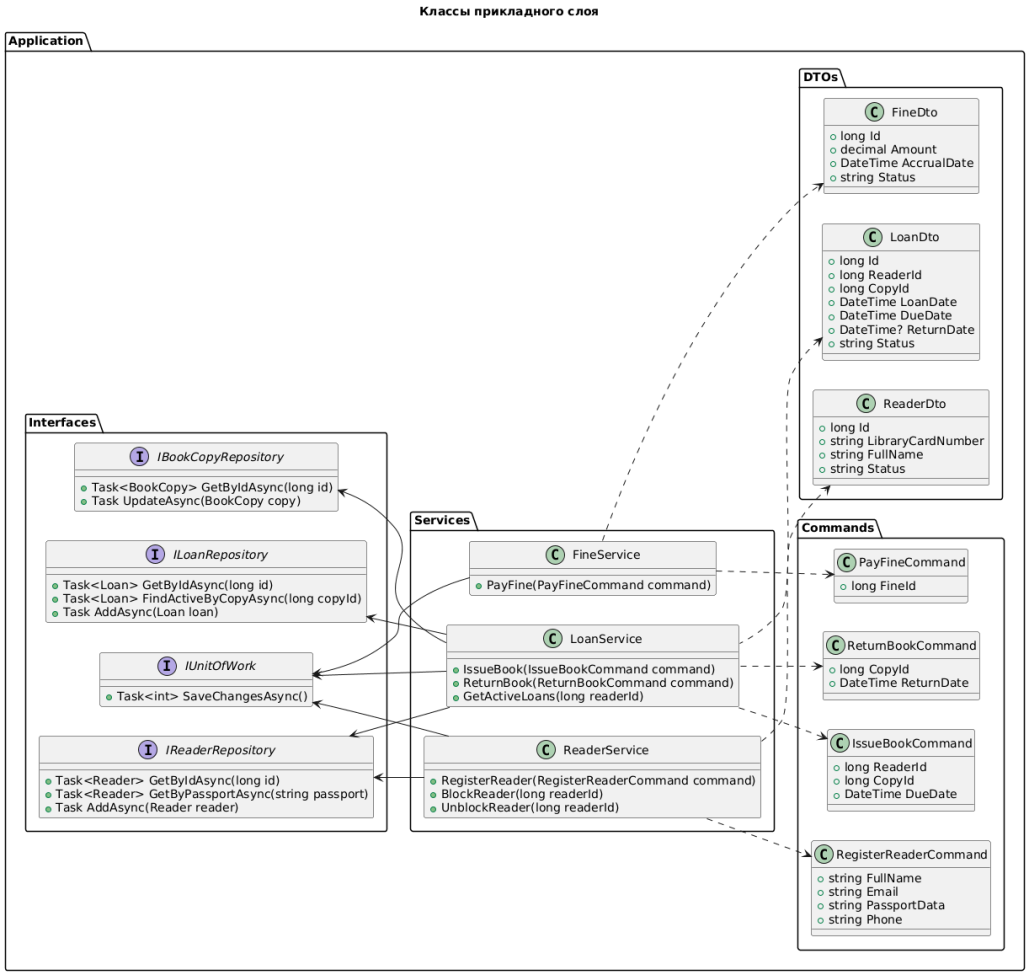

Рисунок 4 – Классы прикладного слоя

1.6 Описание публичных контрактов библиотек

Domain экспортирует сущности, value objects, интерфейсы репозиториев. Изменения в сигнатурах методов сущностей или интерфейсов считаются ломающими.

Application экспортирует сервисы и DTO. Контракты сервисов (интерфейсы) должны быть стабильны; добавление новых методов допустимо, удаление или изменение существующих — ломающее изменение.

Contracts экспортирует DTO для внешних интеграций. Изменение полей DTO — ломающее, требует версионирования (например, через пространства имён v1, v2).

Версионирование осуществляется через сборки (AssemblyVersion) и NuGet пакеты (для возможного использования в микросервисах).

1.7 Тестируемость и изоляция зависимостей

Domain: модульные тесты без внешних зависимостей. Тестируются инварианты, методы сущностей. Для каждой сущности создаётся набор тестов, покрывающих разрешённые и запрещённые переходы состояний.

Application: тестирование с использованием моков репозиториев (через интерфейсы). Проверяется оркестрация, вызовы методов домена, обработка ошибок.

Infrastructure: интеграционные тесты с реальной БД (или in-memory) для проверки работы репозиториев;

Минимальный набор: все тесты домена, основные сценарии приложения.

1.8 Шаблон структуры репозитория и правила именования

Примерная труктура каталогов решения представлена в листинге 1.

Листинг 1 – Структура решения

```
./
├── LibrarySystem.sln
├── src/
│   ├── Domain/
│   │   ├── Entities/
│   │   ├── ValueObjects/
│   │   ├── Enums/
│   │   ├── Interfaces/
│   │   └── Domain.csproj
│   ├── Application/
│   │   ├── Services/
│   │   ├── Commands/
│   │   ├── Queries/
│   │   ├── DTOs/
│   │   ├── Interfaces/
│   │   └── Application.csproj
│   ├── Infrastructure/
│   │   ├── Repositories/
│   │   ├── UnitOfWork/
│   │   ├── Email/
│   │   └── Infrastructure.csproj
│   ├── Integration/
│   │   ├── PaymentGateway/
│   │   └── Integration.csproj
```

```
│   ├── Presentation/
│   │   ├── Controllers/
│   │   └── Presentation.csproj (ASP.NET Core)
│   └── SharedKernel/
│       └── SharedKernel.csproj
├── tests/
│   ├── Domain.UnitTests/
│   ├── Application.UnitTests/
│   └── Infrastructure.IntegrationTests/
└── docs/
    └── diagrams/
```

Правила именования:

– Пространства имён: Company.Library.Domain, Company.Library.Application и т.д.;

– Классы сущностей — в единственном числе (Book, Reader);

– Интерфейсы репозиториев — IBookCopyRepository;

– DTO — BookCopyDto, IssueBookCommand.

1.9 План сопровождения структуры библиотек

1. Новые модули добавляются как отдельные проекты в соответствии с зависимостями.

2. При добавлении новой интеграции создаётся проект в Integration, реализующий интерфейсы из Domain/Application.

3. Рецензирование архитектуры: проверка отсутствия циклических зависимостей, соблюдения направления зависимостей.

4. Выделение отдельной библиотеки оправдано, если:

– она содержит функциональность, которая может быть заменена или переиспользована;

– она инкапсулирует работу с внешней системой;

– она может быть протестирована изолированно.

2 РЕАЛИЗАЦИЯ ФРАГМЕНТА МОДЕЛИ В КОДЕ

2.1 Выбор фрагмента UML модели

Для реализации выбран **агрегат выдачи книги** (Loan) с его окружением: сущности `Loan`, `BookCopy`, `Reader`, а также `Fine` (как опциональная часть). Это центральный элемент сценариев выдачи и возврата. Включает инварианты и бизнес-правила.

Количество классов: 4 сущности, 3 перечисления, плюс вспомогательные типы. Всего около 10—12 типов.

2.2 Правила сопоставления UML конструкций к C#

Таблица 2 – Правила сопоставления UML и C#

UML конструкция	**Реализация в C#**
Класс	class с модификатором public
Абстрактный класс	abstract class
Интерфейс	interface
Перечисление	enum
Атрибут класса (свойство)	public auto-property или поле с методами доступа
Метод	public метод
Ассоциация (направленная)	ссылка на объект (или идентификатор, если агрегат)
Композиция (жёсткая связь)	объект создаётся и удаляется вместе с владельцем; в C# — поле с типом класса, создаётся в конструкторе
Агрегация (слабая)	ссылка через интерфейс или идентификатор
Кратность 1	одиночная ссылка
Кратность 0..*	коллекция (IReadOnlyList, List, HashSet)
Инвариант	проверки в конструкторах и методах, исключения

2.3 Создание структуры проекта

Создадим два проекта в решении:

1. `Library.Domain` — классы домена.
2. `Library.Domain.Tests` — модульные тесты.

Используем .NET 8 (или 6).

2.4 Реализация каркаса классов

Перечисления (Enums)

Листинг 2 – Перечисления

```
namespace Library.Domain.Enums {
    public enum BookCopyStatus {
        Available,
        Loaned,
        Repair,
        WrittenOff
    }
    public enum LoanStatus {
        Active,
        Overdue,
        Returned,
        Lost
    }
    public enum FineStatus {
        Unpaid,
        Paid,
        Waived
    }
    public enum ReaderStatus {
        Active,
        Blocked,
        Deleted
    }
}
```

Класс BookCopy

Листинг 3 – BookCopy

```
using Library.Domain.Enums;
namespace Library.Domain.Entities {
    public class BookCopy {
        // Приватный сеттер для обеспечения инкапсуляции
        public long Id { get; private set; }
        public string InventoryNumber { get; private set; }
        public BookCopyStatus Status { get; private set; }
        public decimal Price { get; private set; }
        public string ShelfLocation { get; private set; }
        // для EF Core
        private BookCopy() { }
        public BookCopy(string inventoryNumber, decimal price, string shelfLocation) {
            if (string.IsNullOrWhiteSpace(inventoryNumber))
                throw new ArgumentException(
                    "Inventory number cannot be empty", nameof(inventoryNumber));
            if (price < 0)
                throw new ArgumentException(
                    "Price cannot be negative", nameof(price));
            InventoryNumber = inventoryNumber;
            Price = price;
```

```
            ShelfLocation = shelfLocation;
            Status = BookCopyStatus.Available;
        }
        public void ChangeStatus(BookCopyStatus newStatus) {
            // Бизнесправила- переходов можно( вынести в отдельный валидатор)
            if (Status == BookCopyStatus.WrittenOff)
                throw new InvalidOperationException(
                    "Cannot change status of a written-off copy");
            if (Status == BookCopyStatus.Loaned && newStatus == BookCopyStatus.Loaned)
                throw new InvalidOperationException("Copy is already loaned");
            Status = newStatus;
        }
        public void Loan() {
            if (Status != BookCopyStatus.Available)
                throw new InvalidOperationException(
                    "Only available copies can be loaned");
            Status = BookCopyStatus.Loaned;
        }
        public void Return() {
            if (Status != BookCopyStatus.Loaned)
                throw new InvalidOperationException(
                    "Only loaned copies can be returned");
            Status = BookCopyStatus.Available;
        }
        public void SendToRepair() {
            if (Status != BookCopyStatus.Available)
                throw new InvalidOperationException(
                    "Only available copies can be sent to repair");
            Status = BookCopyStatus.Repair;
        }
        public void RepairCompleted() {
            if (Status != BookCopyStatus.Repair)
                throw new InvalidOperationException(
                    "Only copies in repair can be marked as repaired");
            Status = BookCopyStatus.Available;
        }
        public void WriteOff() {
            if (Status == BookCopyStatus.WrittenOff)
                throw new InvalidOperationException("Copy is already written off");
            Status = BookCopyStatus.WrittenOff;
        }
    }
}
```

Класс Reader

Листинг 4 – Reader

```
using Library.Domain.Enums;
namespace Library.Domain.Entities {
    public class Reader {
        public long Id { get; private set; }
        public string LibraryCardNumber { get; private set; }
        public string FullName { get; private set; }
        public string Email { get; private set; }
        public string PassportData { get; private set; }
        public ReaderStatus Status { get; private set; }
        private Reader() { }
        public Reader(
            string libraryCardNumber,
            string fullName,
            string email,
            string passportData
        ) {
            if (string.IsNullOrWhiteSpace(libraryCardNumber))
                throw new ArgumentException(
                    "Library card number is required", nameof(libraryCardNumber));
            if (string.IsNullOrWhiteSpace(fullName))
                throw new ArgumentException("Full name is required", nameof(fullName));
            if (string.IsNullOrWhiteSpace(email))
                throw new ArgumentException("Email is required", nameof(email));
            if (string.IsNullOrWhiteSpace(passportData))
                throw new ArgumentException(
                    "Passport data is required", nameof(passportData));
            LibraryCardNumber = libraryCardNumber;
            FullName = fullName;
            Email = email;
            PassportData = passportData;
            Status = ReaderStatus.Active;
        }
        public bool CanBorrow(int activeLoanCount, int maxLoans) {
            return Status == ReaderStatus.Active && activeLoanCount < maxLoans;
        }
        public void Block() {
            if (Status == ReaderStatus.Deleted)
                throw new InvalidOperationException(
                    "Cannot block deleted reader");
            Status = ReaderStatus.Blocked;
        }
        public void Unblock() {
            if (Status != ReaderStatus.Blocked)
                throw new InvalidOperationException(
                    "Only blocked reader can be unblocked");
            Status = ReaderStatus.Active;
        }
        public void Delete() {
            // Доменное правило: нельзя удалить,
            // если есть активные выдачи проверяется( на уровне приложения)
            Status = ReaderStatus.Deleted;
        }
    }
}
```

Класс Fine

Листинг 5 – Fine

```
using Library.Domain.Enums;
namespace Library.Domain.Entities {
    public class Fine {
        public long Id { get; private set; }
        public decimal Amount { get; private set; }
        public DateTime AccrualDate { get; private set; }
        public DateTime? PaymentDate { get; private set; }
        public FineStatus Status { get; private set; }
        private Fine() { }
        public Fine(decimal amount, DateTime accrualDate) {
            if (amount <= 0)
                throw new ArgumentException(
                    "Fine amount must be positive", nameof(amount));
            Amount = amount;
            AccrualDate = accrualDate;
            Status = FineStatus.Unpaid;
        }
        public void Pay(DateTime paymentDate) {
            if (Status != FineStatus.Unpaid)
                throw new InvalidOperationException(
                    "Only unpaid fines can be paid");
            PaymentDate = paymentDate;
            Status = FineStatus.Paid;
        }
        public void Waive() {
            if (Status != FineStatus.Unpaid)
                throw new InvalidOperationException(
                    "Only unpaid fines can be waived");
            Status = FineStatus.Waived;
        }
    }
}
```

Класс Loan (агрегат)

Листинг 6 – Loan

```
using Library.Domain.Enums;
namespace Library.Domain.Entities {
    public class Loan {
        public long Id { get; private set; }
        public long ReaderId { get; private set; }
        public long BookCopyId { get; private set; }
        public DateTime LoanDate { get; private set; }
        public DateTime DueDate { get; private set; }
        public DateTime? ReturnDate { get; private set; }
        public LoanStatus Status { get; private set; }
        // Навигационные свойства для( ORM, но в домене не обязательны)
        public virtual Reader Reader { get; private set; }
        public virtual BookCopy BookCopy { get; private set; }
        public virtual Fine Fine { get; private set; }
        private Loan() { }
```

```
        public Loan(
            Reader reader, BookCopy copy, DateTime loanDate, DateTime dueDate
        ) {
            if (reader == null)
                throw new ArgumentNullException(nameof(reader));
            if (copy == null)
                throw new ArgumentNullException(nameof(copy));
            if (loanDate > dueDate)
                throw new ArgumentException("Loan date cannot be after due date");
            if (copy.Status != BookCopyStatus.Available)
                throw new InvalidOperationException(
                    "Cannot loan a copy that is not available");
            if (reader.Status != ReaderStatus.Active)
                throw new InvalidOperationException("Reader is not active");
            ReaderId = reader.Id;
            BookCopyId = copy.Id;
            LoanDate = loanDate;
            DueDate = dueDate;
            Status = LoanStatus.Active;
            // Изменяем статус копии
            copy.Loan();
        }
        public Fine ReturnBook(DateTime returnDate, decimal dailyFineRate) {
            if (Status != LoanStatus.Active && Status != LoanStatus.Overdue)
                throw new InvalidOperationException(
                    "Only active or overdue loans can be returned");
            ReturnDate = returnDate;
            BookCopy.Return(); // возвращаем копию в Available
            if (returnDate > DueDate) {
                var overdueDays = (returnDate - DueDate).Days;
                var fineAmount = overdueDays * dailyFineRate;
                Fine = new Fine(fineAmount, returnDate);
                Status = LoanStatus.Returned;
                return Fine;
            } else {
                Status = LoanStatus.Returned;
                return null;
            }
        }
        public void MarkOverdue() {
            if (Status != LoanStatus.Active)
                throw new InvalidOperationException(
                    "Only active loans can become overdue");
            Status = LoanStatus.Overdue;
        }
        public void MarkLost() {
            if (Status != LoanStatus.Active && Status != LoanStatus.Overdue)
                throw new InvalidOperationException(
                    "Only active or overdue loans can be marked lost");
            Status = LoanStatus.Lost;
            BookCopy.WriteOff(); // или другой метод утери
        }
    }
}
```

2.5 Реализация инвариантов и бизнес ограничений

Все инварианты реализованы через проверки в конструкторах и методах, выбрасываются исключения с понятным сообщением. Например, в `Loan.ReturnBook` проверяется допустимость возврата; при просрочке создаётся Fine.

2.6 Реализация отношений между объектами

`Loan` содержит ссылки на `Reader` и `BookCopy` (навигационные свойства). В конструкторе они передаются и сохраняются.

Для упрощения идентификаторы также хранятся отдельно (для случая, когда объекты не загружены).

Коллекции не используются в этом фрагменте, но если бы были, то возвращались бы как `IReadOnlyList`.

2.7 Подготовка модульных тестов

Создадим проект тестов и добавим ссылку на Domain.

Примеры тестов (xUnit) показаны в листинге 7.

Листинг 7 – LoanTests

```
using Library.Domain.Entities;
using Library.Domain.Enums;
using System;
using Xunit;
namespace Library.Domain.Tests {
    public class LoanTests {
        [Fact]
        public void CreateLoan_ValidData_SetsStatusActive() {
            // Arrange
            var reader = new Reader(
                "C001", "John Doe", "john@test.com", "AB123456");
            var copy = new BookCopy("INV001", 500, "A1");
            // Act
            var loan = new Loan(
                reader, copy, DateTime.Today, DateTime.Today.AddDays(14));
            // Assert
            Assert.Equal(LoanStatus.Active, loan.Status);
            Assert.Equal(BookCopyStatus.Loaned, copy.Status);
        }
        [Fact]
        public void CreateLoan_WithUnavailableCopy_Throws() {
```

```
            var reader = new Reader(
                "C001", "John Doe", "john@test.com", "AB123456");
            var copy = new BookCopy("INV001", 500, "A1");
            copy.Loan(); // делаем недоступной
            Assert.Throws<InvalidOperationException>(() => new Loan(
                reader, copy, DateTime.Today, DateTime.Today.AddDays(14)
            ));
        }
        [Fact]
        public void ReturnBook_OnTime_NoFine() {
            var reader = new Reader(
                "C001", "John Doe", "john@test.com", "AB123456");
            var copy = new BookCopy("INV001", 500, "A1");
            var loan = new Loan(
                reader, copy, DateTime.Today, DateTime.Today.AddDays(14));
            var fine = loan.ReturnBook(DateTime.Today.AddDays(10), 10);
            Assert.Null(fine);
            Assert.Equal(LoanStatus.Returned, loan.Status);
            Assert.Equal(BookCopyStatus.Available, copy.Status);
        }
        [Fact]
        public void ReturnBook_WithOverdue_CreatesFine() {
            var reader = new Reader(
                "C001", "John Doe", "john@test.com", "AB123456");
            var copy = new BookCopy("INV001", 500, "A1");
            var loan = new Loan(
                reader, copy, DateTime.Today.AddDays(-20), DateTime.Today.AddDays(-6));
            var fine = loan.ReturnBook(DateTime.Today, 10);
            Assert.NotNull(fine);
            Assert.Equal(60, fine.Amount); // 6 дней * 10
            Assert.Equal(FineStatus.Unpaid, fine.Status);
            Assert.Equal(LoanStatus.Returned, loan.Status);
        }
        [Fact]
        public void MarkOverdue_FromActive_ChangesStatus() {
            var reader = new Reader(
                "C001", "John Doe", "john@test.com", "AB123456");
            var copy = new BookCopy("INV001", 500, "A1");
            var loan = new Loan(
                reader, copy, DateTime.Today.AddDays(-20), DateTime.Today.AddDays(-6));
            loan.MarkOverdue();
            Assert.Equal(LoanStatus.Overdue, loan.Status);
        }
        [Fact]
        public void MarkOverdue_FromReturned_Throws() {
            var reader = new Reader("C001", "John Doe", "john@test.com", "AB123456");
            var copy = new BookCopy("INV001", 500, "A1");
            var loan = new Loan(
                reader, copy, DateTime.Today.AddDays(-20), DateTime.Today.AddDays(-6));
            loan.ReturnBook(DateTime.Today, 10);
            Assert.Throws<InvalidOperationException>(() => loan.MarkOverdue());
        }
    }
}
```

Аналогичные тесты для BookCopy, Reader, Fine.

2.8 Таблица соответствия UML и кода

Таблица 3 – Соответствие UML и кода

Элемент UML	**Элемент кода**	**Примечания**
Класс `BookCopy`	`Library.Domain .Entities.BookCopy`	Реализован с инкапсуляцией статуса
Атрибут `inventoryNumber`	свойство `InventoryNumber`	private set
Атрибут `status: BookCopyStatus`	свойство `Status`	enum
Метод `changeStatus(newStatus)`	методы `Loan()`, `Return()`, `SendToRepair()` и т.д.	Вместо общего changeStatus сделаны специализированные методы
Класс `Reader`	`Reader`	
Класс `Loan`	`Loan`	
Ассоциация `Loan -> Reader`	свойство `Reader` и `ReaderId`	
Ассоциация `Loan -> BookCopy`	свойство `BookCopy` и `BookCopyId`	
Композиция `Loan -> Fine` (0..1)	свойство `Fine`	Fine создаётся внутри метода ReturnBook
Инвариант «книга не может быть выдана дважды»	проверка в `BookCopy.Loan()`	
Инвариант «читатель должен быть активен»	проверка в конструкторе `Loan`	
Переходы состояний Loan	методы `MarkOverdue()`, `ReturnBook()`, `MarkLost()`	

Отклонения: общий метод `ChangeStatus` заменён на конкретные методы для каждого допустимого перехода, что повышает безопасность и читаемость.

ЗАКЛЮЧЕНИЕ

Разработанные модели и код полностью соответствуют требованиям практических занятий №11 и №12. Структура библиотек обеспечивает чёткое разделение ответственности, изоляцию домена и тестируемость. Реализованный фрагмент доменной модели демонстрирует корректное отображение UML в C# и соблюдение бизнес-правил. Полученные артефакты готовы к использованию в следующих этапах разработки (реализация прикладного слоя, инфраструктуры и представления).